Informe: Buscador seguro con PDO y Consultas Preparadas

Backend en PHP (PDO) + Frontend en JavaScript Vanilla | Base de datos: clase_backend | 13/10/2026

1. De qué se trata el trabajo

El objetivo fue desarrollar un módulo de búsqueda de productos con un backend seguro en PHP. El punto central es evitar la **inyección SQL**, un ataque en el que un usuario introduce código SQL en el campo de búsqueda para alterar la consulta y obtener datos que no debería ver.

El proyecto combina:

- **Frontend:** un formulario en `index.html` y `app.js`, que envía la búsqueda al servidor con `fetch` y dibuja los resultados sin recargar la página.
- **Backend:** un endpoint en PHP que consulta la tabla `productos` mediante PDO y devuelve los datos en formato JSON.

2. La vulnerabilidad detectada

La versión original armaba la consulta **concatenando** directamente lo que escribía el usuario:

```
$sql = "SELECT * FROM productos WHERE nombre LIKE '%" . $q . "%'";
```

Al concatenar, el dato del usuario se mezcla con el código SQL. Si se ingresa la cadena `' OR '1'='1`, la comilla cierra el texto y el `OR` pasa a ser un operador lógico. La condición queda siempre verdadera y el motor devuelve **toda la tabla**.

3. La solución aplicada

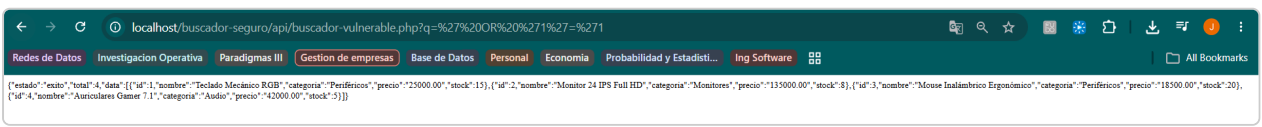
Se reescribió el endpoint usando una **consulta preparada** con PDO, que separa el SQL de los datos:

```
$stmt = $pdo->prepare(
    "SELECT id, nombre, categoria, precio, stock
     FROM productos WHERE nombre LIKE :termino ORDER BY nombre ASC");
$stmt->execute([':termino' => '%' . $q . '%']);
```

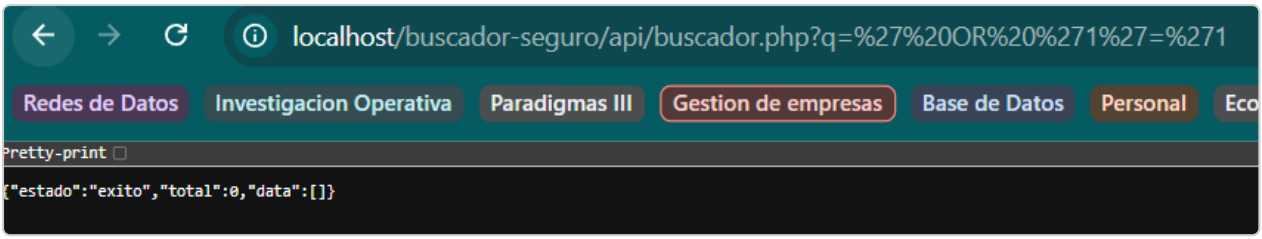
La consulta viaja primero con el marcador `:termino` (sin datos) y el motor fija su estructura. Recién después se envía el valor del usuario, por separado, y se trata siempre como texto, nunca como código. Además la conexión usa `ATTR_EMULATE_PREPARES => false` para que la preparación sea real en el servidor.

4. Comprobación (antes y después)

Se probó la misma inyección `' OR '1'='1` en ambas versiones:



Buscador vulnerable: la inyección devuelve **total: 4** (toda la tabla). El ataque funciona.



Buscador seguro: la misma inyección devuelve **total: 0** y lista vacía. El ataque queda neutralizado.

5. Conclusión

El uso de consultas preparadas con PDO resolvió la vulnerabilidad de raíz. La entrada del usuario puede cambiar *qué* se busca, pero nunca *cómo* está armada la consulta. Como medida complementaria conviene usar un usuario de base de datos con permisos mínimos y no mostrar al usuario los mensajes de error del motor.